PANDUAN OPERASIONAL

# Instalasi Server Baru

Dari server Ubuntu kosong sampai layanan chat & komunitas bisa diakses publik lewat domain dengan HTTPS.

**Ringkasnya, tiga langkah**

1. Dari laptop, kirim script ke server: `scp scripts/server-install.sh <user>@<IP-server>:~/install.sh`
2. Masuk ke server ( `ssh <user>@<IP-server>` ), jalankan `bash install.sh` , dan jawab pertanyaannya
3. Selesaikan pengaturan di Rendezvous Hub (bagian 5)

Sumber: docs/server-install.md · Dibuat 4 November 2026

Panduan memasang server sosial di server Ubuntu baru sampai API bisa diakses publik lewat domain dengan HTTPS. Sebagian besar langkah dikerjakan satu script, `scripts/server-install.sh`, yang dikirim ke server dengan `scp` sebelum repo di-clone di sana. Isinya juga dimuat di Lampiran.

Hasil akhirnya:

- `https://<domain API>` untuk aplikasi OLT (REST, Realtime, login, unggah media), sertifikat Let's Encrypt otomatis.
- Rendezvous Hub di `https://<domain Hub>`, hanya dari IP kantor/VPN (opsional).
- Backup berjalan ke folder di disk terpisah setelah diatur dari Hub.

Rujukan lain: operations.md (operasional), runbook-gangguan.md (gangguan), PRODUCTION.md (detail produksi), dev-server.md (spek dan mode dev).

## 1. Yang harus ada sebelum mulai

| | Dari | Contoh |
|---|---|---|
| Server Ubuntu 24.04, 4 vCPU / 8 GB / 100 GB SSD (+ disk backup terpisah, mis. `/mnt/backup`) | Vendor IT | — |
| IP publik server (atau port forwarding 80/443 dari IP publik kantor) | Vendor IT | `203.0.113.10` |
| A record domain API ke IP itu | Vendor IT / pengelola DNS | `social-api.perusahaan.co.id` |
| A record domain Hub ke IP yang sama (opsional) | Vendor IT / pengelola DNS | `hub.perusahaan.co.id` |
| Firewall penyedia: 80 dan 443 dari internet, 22 hanya dari IP admin | Vendor IT | — |
| Akses HTTPS keluar dari server ke `database.clamav.net` (signature antivirus, ±300 MB saat start pertama lalu update harian) | Vendor IT | — |
| User SSH dengan `sudo`, login dengan SSH key | Vendor IT | — |
| JWKS (kunci **publik**) penanda tangan token sosial | Tim backend OLT (olt-integration.md) | `{"keys":[{"kty":"OKP",...}]}` |
| Email tim IT (notifikasi sertifikat) | Anda | `it@perusahaan.co.id` |
| IP publik kantor / subnet VPN untuk Hub | Tim IT | `198.51.100.7/32` |

| | Dari | Contoh |
|---|---|---|
| Akses admin repo GitHub `chat_service_supabase`, dan `rendezvous_hub` bila Hub dipasang (untuk menambah deploy key) | Anda | — |

Tanpa JWKS dari OLT, mode produksi tidak bisa dipakai (kunci dummy ditolak). Selama JWKS belum ada, pakai mode dev di bagian 7.

## 2. Menjalankan script

1. Kirim script ke server. Cara paling aman: dari laptop yang sudah punya repo ini (PowerShell atau terminal di folder repo):

   ```
   scp scripts/server-install.sh <user>@<IP-server>:~/install.sh
   ```

   Cara lain bila repo tidak ada di laptop: setelah masuk ke server (langkah 2), jalankan `nano install.sh`, tempel isi file dari GitHub, lalu Ctrl+O, Enter, Ctrl+X. Jangan menyalin script dari PDF: baris panjang yang terlipat bisa ikut tersalin sebagai baris baru dan merusak script.

2. Masuk ke server:

   ```
   ssh <user>@<IP-server>
   ```

3. Jalankan di server:

   ```
   bash install.sh
   ```

Script menanyakan beberapa hal dan berhenti dengan pesan jelas bila ada yang belum siap. Aman dijalankan ulang: langkah yang sudah selesai dilewati, isian sebelumnya menjadi nilai bawaan.

| Langkah | Yang dilakukan | Yang ditanyakan |
|---|---|---|
| 1 | Update sistem, paket dasar, update keamanan otomatis, NTP | — |
| 2 | Pasang Docker dan compose dari repo resmi Docker | — |
| 3 | Swap 4 GB bila RAM < 8 GB | — |
| 4 | `ufw`: SSH, 80, 443 | — |
| 5 | Deploy key GitHub (baca saja), clone repo ke `~/chat_service_supabase` | Tambahkan deploy key yang ditampilkan di GitHub, lalu Enter |

| Langkah | Yang dilakukan | Yang ditanyakan |
|---|---|---|
| 6 | `make gen-secrets`, isi `docker/.env` | Domain API, email, domain Hub dan IP kantor (opsional), folder backup |
| 7 | Bila domain Hub diisi: deploy key kedua (`~/.ssh/github_deploy_hub`, alias SSH `github-hub`), clone `rendezvous_hub` ke `~/rendezvous_hub`; saat dijalankan ulang cukup `git pull` | Tambahkan deploy key kedua di repo `rendezvous_hub`, lalu Enter |
| 8 | Simpan JWKS OLT ke `docker/secrets/olt_jwks`, tolak kunci dev atau kunci privat | Tempel JWKS, Enter, Ctrl+D |
| 9 | Cek DNS domain mengarah ke server | Konfirmasi IP |
| 10 | `make up` profil `single`, akun Super Admin pertama, verifikasi HTTPS | Email dan nama Super Admin |

GitHub menolak satu deploy key dipakai di dua repo, jadi Hub memakai kunci sendiri. Image Hub dibangun dari `~/rendezvous_hub` (letaknya bersebelahan dengan `~/chat_service_supabase`). Bila `HUB_APP_DIR` diatur ke folder lain, script mengisi `HUB_DIR` di `docker/.env`.

Di akhir, script mencetak URL API, anon key, dan alamat Hub. Password sementara Super Admin hanya tampil **sekali**; catat sebelum menutup terminal.

Waktu: ±15–30 menit, sebagian besar untuk mengunduh dan build image pertama kali.

## 3. Isi script

Isi lengkap script dimuat di **Lampiran** di akhir dokumen ini untuk dibaca. Untuk dipakai, kirim file `scripts/server-install.sh` dari repo dengan `scp` (bagian 2), jangan disalin dari PDF.

## 4. Cek setelah selesai

Script sudah memeriksa tiga hal di bawah. Ulangi kapan saja:

```
cd ~/chat_service_supabase
make ps                                                    # semua healthy
curl -s -H "apikey: $(cat docker/secrets/anon_key)" https://<domain API>/bridge/v1/health
                                                           # "olt_mode":"jwt", "status":"ok"
curl -sI http://<domain API>/ | head -1                    # 301/308 ke https
curl -s -o /dev/null -w '%{http_code}\n' https://<domain API>/   # 404: hanya path API yang dibuka
```

Dari laptop di luar kantor: `nc -zv <IP-server> 5432` harus gagal, dan `https://<domain Hub>` harus 403.

## 5. Setelah server jalan

1. **Login Hub** ( `https://<domain Hub>` , dari kantor/VPN) dengan email Super Admin dan password sementara, lalu ganti password.
2. **Pengaturan** → **Backup**: pilih tujuan (Folder lokal di server, atau S3), buat password enkripsi, simpan. Tekan **Uji restore sekarang** setelah backup penuh pertama selesai.
3. **Pengaturan** → **Alert**: isi Telegram/email, lalu **Kirim uji**. Tanpa kanal ini, alert gangguan tidak terkirim ke siapa pun.
4. **Pengaturan** → **Push notification**: unggah kunci Firebase (project Firebase aplikasi OLT), bila push dipakai.
5. **Pengaturan** → **Akun CS**: buat akun staf lain.
6. **Simpan di luar server** (password manager / brankas): `docker/.env` , folder `docker/secrets/` , password backup, salinan `pgsodium_root.key` (backup).
7. **Data untuk tim aplikasi OLT**: `supabaseUrl` = `https://<domain API>` , `anonKey` = hasil `make anon-key` , versi library `v1.11.0` (panduan integrasi library).
8. Uji dari aplikasi: login, buka feed, kirim chat antar dua akun.

## 6. Bila ada masalah saat instalasi

| Gejala | Penyebab umum | Tindakan |
|---|---|---|
| "does not resolve to any IP yet" | A record belum dibuat atau belum menyebar | Minta tim IT cek DNS, tunggu beberapa menit, jalankan ulang |
| Health tidak menjawab, `make logs s=traefik` berisi error ACME | Port 80 tertutup, DNS salah, atau batas Let's Encrypt | Pastikan 80/443 terbuka dari internet dan DNS benar, lalu `make up` |
| `docker/.env still contains CHANGE_ME` atau `is required` | Isian `.env` kurang | Jalankan ulang script, atau edit `docker/.env` lalu `make up` |
| `JWKS rejected` | Kunci dev, kunci privat, atau format salah | Minta ulang JWKS **publik** dari tim OLT |
| Clone terus gagal | Deploy key belum ditambahkan / repo lain | Periksa GitHub → Settings → Deploy keys. Kunci `github_deploy` untuk `chat_service_supabase` , kunci `github_deploy_hub` untuk `rendezvous_hub` |
| `Rendezvous Hub checkout not found at ...` | Domain Hub diisi tetapi `~/rendezvous_hub` belum ada | Jalankan ulang script (langkah 7), atau isi `HUB_DIR` di `docker/.env` |
| Hub 403 dari kantor | IP kantor tidak ada di `VPN_SOURCE_RANGES` | Cek IP publik kantor, perbaiki `docker/.env` , `make up` |

| Gejala | Penyebab umum | Tindakan |
|---|---|---|
| `olt_mode` bukan `jwt` | Konfigurasi auth-bridge | `make logs s=auth-bridge` |
| `make up` habis waktu menunggu `clamav`, atau Hub menolak dokumen/video chat CS dengan "Virus check is unavailable" | Start pertama clamav masih mengunduh signature (±300 MB) atau server tidak bisa mengakses `database.clamav.net` | `make logs s=clamav` ; `make up` sudah menunggu sampai 15 menit; bila tetap habis waktu, tunggu sampai "socket found, clamd started", lalu `make up` lagi. clamd butuh ±1,1–1,6 GB RAM (lihat PRODUCTION.md) |
| Login dari aplikasi selalu "Sesi berakhir" | `iss` / `aud` /jam server OLT tidak cocok | `docker logs olt-social-auth-bridge-1 --since 15m` (field `reason`) |

## 7. Belum ada JWKS OLT: mode dev sementara

Mode ini memakai OLT dummy (login uji `budi_santoso` / `olt12345` dan lainnya), tanpa domain dan HTTPS, diakses lewat SSH tunnel. Langkah 1–5 script tetap bisa dipakai: jalankan script, lalu berhenti (Ctrl+C) saat ditanya domain. Sesudahnya:

```
cd ~/chat_service_supabase
make up DUMMY_OLT=1
make seed
make smoke-dummy        # semua ok
make cs-staff           # akun staf dev untuk Hub
```

Hub di mode dev butuh clone `rendezvous_hub` di `~/rendezvous_hub` (tanpa itu `make up` berjalan tanpa `cs-console`). Karena langkah 7 script hanya berjalan bila domain Hub diisi, clone manual dengan deploy key kedua:

```
ssh-keygen -q -t ed25519 -N "" -C "deploy-hub@$(hostname)" -f ~/.ssh/github_deploy_hub
printf 'Host github-hub\n  HostName github.com\n  IdentityFile ~/.ssh/github_deploy_hub\n
IdentitiesOnly yes\n' >> ~/.ssh/config
cat ~/.ssh/github_deploy_hub.pub     # tambahkan di repo rendezvous_hub -> Settings -> Deploy keys
(baca saja)
git clone git@github-hub:dayat123123/rendezvous_hub.git ~/rendezvous_hub
cd ~/chat_service_supabase && make cs-console-react
```

Dari laptop:

```
ssh -L 8000:127.0.0.1:8000 -L 8090:127.0.0.1:8090 -L 8085:127.0.0.1:8085 <user>@<IP-server>
```

Selama terminal itu terbuka: Hub `http://localhost:8085`, API `http://localhost:8000`, login OLT dummy `http://localhost:8090`. Setelah JWKS OLT diterima: `make down`, lalu jalankan script lagi dan lanjutkan sampai selesai.

## 8. Sehari-hari

```
cd ~/chat_service_supabase
make ps                  # status
make logs s=rest         # log satu layanan (Ctrl+C keluar)
git pull && make up      # update versi, di luar jam bursa, setelah make backup
```

Bila Hub dipasang, perbarui juga repo Hub sebelum `make up`: `git -C ~/rendezvous_hub pull --ff-only` (atau jalankan ulang script, yang menarik kedua repo).

`PROFILE=single` (dan `CS=1` bila Hub dipasang) sudah disimpan script di `~/.bashrc`, jadi perintah `make` langsung memakai profil produksi. Jangan pernah menjalankan `make reset` atau `docker volume rm` di server ini.

## Lampiran — Isi script `install.sh`

> **Untuk dibaca, bukan disalin.** Baris panjang di bawah terlipat agar muat di kertas; bila disalin dari PDF, lipatannya bisa merusak script. Kirim file aslinya: `scp scripts/server-install.sh <user>@<IP-server>:~/install.sh`

```
#!/usr/bin/env bash
set -euo pipefail

REPO_SSH="${REPO_SSH:-git@github.com:dayat123123/chat_service_supabase.git}"
APP_DIR="${APP_DIR:-$HOME/chat_service_supabase}"
HUB_REPO_SSH="${HUB_REPO_SSH:-git@github-hub:dayat123123/rendezvous_hub.git}"
HUB_APP_DIR="${HUB_APP_DIR:-$HOME/rendezvous_hub}"
SCRIPT_PATH="$(readlink -f "$0")"

say() { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
ok() { printf '\033[32m  ok  %s\033[0m\n' "$*"; }
warn() { printf '\033[33m  !   %s\033[0m\n' "$*"; }
die() { printf '\033[31m  x   %s\033[0m\n' "$*" >&2; exit 1; }
ask() {
  local prompt=$1 default=${2:-} reply
  read -r -p "  $prompt${default:+ [$default]}: " reply
  printf '%s' "${reply:-$default}"
}
confirm() {
  local reply
  read -r -p "  $1 [y/N]: " reply
  [[ $reply =~ ^[Yy]$ ]]
}
env_get() {
  [ -f "$APP_DIR/docker/.env" ] || return 0
  sed -n "s/^$1=//p" "$APP_DIR/docker/.env" | tail -1
}
env_set() {
  local file="$APP_DIR/docker/.env" tmp
  tmp=$(mktemp)
  awk -v k="$1" -v v="$2" '
    $0 ~ "^"k"=" { if (!done) print k"="v; done = 1; next }
    { print }
    END { if (!done) print k"="v }
  ' "$file" > "$tmp"
  cat "$tmp" > "$file"
  rm -f "$tmp"
}
bashrc_export() {
  grep -qx "export $1=$2" "$HOME/.bashrc" 2>/dev/null || echo "export $1=$2" >> "$HOME/.bashrc"
}

[ "$(id -u)" -ne 0 ] || die "Run as a regular user with sudo, not as root."
. /etc/os-release
[ "${ID:-}" = ubuntu ] || die "This script is for Ubuntu (detected: ${ID:-unknown})."
sudo -v

say "1/10 System packages, automatic security updates, clock"
sudo apt-get update -qq
```

```
sudo DEBIAN_FRONTEND=noninteractive apt-get -y -qq upgrade
sudo DEBIAN_FRONTEND=noninteractive apt-get -y -qq install git make curl ca-certificates unattended-upgrades
ufw python3
echo 'unattended-upgrades unattended-upgrades/enable_auto_updates boolean true' | sudo debconf-set-selections
sudo DEBIAN_FRONTEND=noninteractive dpkg-reconfigure -f noninteractive unattended-upgrades
sudo timedatectl set-ntp true
ok "packages installed, automatic security updates on, NTP on"

say "2/10 Docker"
if ! command -v docker >/dev/null 2>&1; then
  sudo install -m 0755 -d /etc/apt/keyrings
  sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
  echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu ${VERSION_CODENAME} stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
  sudo apt-get update -qq
  sudo DEBIAN_FRONTEND=noninteractive apt-get -y -qq install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
fi
sudo systemctl enable --now docker > /dev/null
if ! id -nG "$USER" | grep -qw docker; then
  sudo usermod -aG docker "$USER"
fi
if ! docker info > /dev/null 2>&1; then
  ok "user $USER added to the docker group; continuing with the new group"
  exec sg docker -c "bash '$SCRIPT_PATH'"
fi
compose_version=$(docker compose version --short | sed 's/^v//')
printf '%s\n2.24.0\n' "$compose_version" | sort -V -C -r || die "docker compose $compose_version is too old
(needs 2.24 or newer)."
ok "docker $(docker version --format '{{.Server.Version}}'), compose $compose_version"

say "3/10 Swap (only when RAM < 8 GB)"
mem_kb=$(awk '/MemTotal/ {print $2}' /proc/meminfo)
if [ "$mem_kb" -lt 7500000 ] && ! swapon --show | grep -q .; then
  sudo fallocate -l 4G /swapfile
  sudo chmod 600 /swapfile
  sudo mkswap /swapfile > /dev/null
  sudo swapon /swapfile
  grep -q '^/swapfile ' /etc/fstab || echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab > /dev/null
  ok "4 GB swap created"
else
  ok "no extra swap needed"
fi

say "4/10 Server firewall (ufw, extra layer)"
ssh_port=$(sudo sshd -T 2>/dev/null | awk '/^port / && !seen {print $2; seen = 1}' || true)
ssh_port=${ssh_port:-22}
sudo ufw allow "$ssh_port/tcp" > /dev/null
sudo ufw allow 80/tcp > /dev/null
sudo ufw allow 443/tcp > /dev/null
sudo ufw --force enable > /dev/null
ok "ufw: $ssh_port (SSH), 80, 443"
warn "Docker container ports bypass ufw; the main restriction stays in the hosting provider's firewall"

say "5/10 Code from GitHub"
if [ -d "$APP_DIR/.git" ]; then
  git -C "$APP_DIR" pull --ff-only
  ok "code updated in $APP_DIR"
```

```
else
  mkdir -p "$HOME/.ssh"
  chmod 700 "$HOME/.ssh"
  [ -f "$HOME/.ssh/github_deploy" ] || ssh-keygen -q -t ed25519 -N "" -C "deploy@$(hostname)" -f
"$HOME/.ssh/github_deploy"
  grep -q 'IdentityFile ~/.ssh/github_deploy' "$HOME/.ssh/config" 2>/dev/null \
    || printf 'Host github.com\n  IdentityFile ~/.ssh/github_deploy\n  IdentitiesOnly yes\n' >>
"$HOME/.ssh/config"
  chmod 600 "$HOME/.ssh/config"
  ssh-keygen -F github.com > /dev/null 2>&1 || ssh-keyscan -t ed25519 github.com >> "$HOME/.ssh/known_hosts"
2>/dev/null
  until git clone -q "$REPO_SSH" "$APP_DIR" 2>/dev/null; do
    echo
    echo "  Add this deploy key on GitHub:"
    echo "  repo chat_service_supabase -> Settings -> Deploy keys -> Add deploy key"
    echo "  (any title, do NOT tick \"Allow write access\")"
    echo
    cat "$HOME/.ssh/github_deploy.pub"
    echo
    read -r -p "  Press Enter once the deploy key is added... " _
  done
  ok "code in $APP_DIR"
fi
cd "$APP_DIR"

say "6/10 Configuration (docker/.env)"
make -s gen-secrets
domain=$(ask "API domain (e.g. social-api.company.co.id)" "$(env_get PROXY_DOMAIN)")
[[ $domain =~ ^[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ ]] || die "Invalid domain: $domain"
acme_email=$(ask "IT team email for HTTPS certificates" "$(env_get ACME_EMAIL)")
[[ $acme_email == *@*.* ]] || die "Invalid email: $acme_email"
hub_domain=$(ask "Rendezvous Hub domain (leave empty if none yet)" "$(env_get CS_CONSOLE_DOMAIN)")
vpn_ranges=""
if [ -n "$hub_domain" ]; then
  [[ $hub_domain =~ ^[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ ]] || die "Invalid Hub domain: $hub_domain"
  vpn_ranges=$(ask "IPs/ranges allowed to open the Hub, comma separated (e.g. 203.0.113.10/32)" "$(env_get
VPN_SOURCE_RANGES)")
  [[ $vpn_ranges =~ ^[0-9./,\ ]+$ ]] || die "Invalid Hub IP list: $vpn_ranges"
  [[ $vpn_ranges != *0.0.0.0/0* ]] || die "0.0.0.0/0 opens the Hub to the whole internet. Enter office/VPN IPs
only."
fi
backup_default=$(env_get BACKUP_LOCAL_REPO_DIR)
[ -n "$backup_default" ] || { [ -d /mnt/backup ] && backup_default=/mnt/backup/restic; }
backup_dir=$(ask "Local backup folder on a separate disk (empty = backups/restic in the repo)"
"$backup_default")

env_set PROXY_DOMAIN "$domain"
env_set SUPABASE_PUBLIC_URL "https://$domain"
env_set API_EXTERNAL_URL "https://$domain/auth/v1"
env_set ACME_EMAIL "$acme_email"
env_set HEALER_DOCKER_GID "$(getent group docker | cut -d: -f3)"
if [ -n "$hub_domain" ]; then
  env_set CS_CONSOLE_DOMAIN "$hub_domain"
  env_set VPN_SOURCE_RANGES "$vpn_ranges"
fi
if [ -n "$backup_dir" ]; then
  [[ $backup_dir == /* ]] || die "The backup folder must be an absolute path: $backup_dir"
  sudo install -d -o 10001 -g 10001 -m 700 "$backup_dir"
  env_set BACKUP_LOCAL_REPO_DIR "$backup_dir"
```

```
fi
ok "docker/.env updated"

say "7/10 Rendezvous Hub code (only when the Hub is installed)"
if [ -d "$HUB_APP_DIR/.git" ]; then
  git -C "$HUB_APP_DIR" pull --ff-only
  ok "Hub code updated in $HUB_APP_DIR"
elif [ -n "$hub_domain" ]; then
  mkdir -p "$HOME/.ssh"
  chmod 700 "$HOME/.ssh"
  [ -f "$HOME/.ssh/github_deploy_hub" ] || ssh-keygen -q -t ed25519 -N "" -C "deploy-hub@$(hostname)" -f
"$HOME/.ssh/github_deploy_hub"
  grep -qx 'Host github-hub' "$HOME/.ssh/config" 2>/dev/null \
    || printf 'Host github-hub\n  HostName github.com\n  IdentityFile ~/.ssh/github_deploy_hub\n
IdentitiesOnly yes\n' >> "$HOME/.ssh/config"
  chmod 600 "$HOME/.ssh/config"
  ssh-keygen -F github.com > /dev/null 2>&1 || ssh-keyscan -t ed25519 github.com >> "$HOME/.ssh/known_hosts"
2>/dev/null
  until git clone -q "$HUB_REPO_SSH" "$HUB_APP_DIR" 2>/dev/null; do
    echo
    echo "  Add this second deploy key on GitHub (GitHub needs a separate deploy key per repo):"
    echo "  repo rendezvous_hub -> Settings -> Deploy keys -> Add deploy key"
    echo "  (any title, do NOT tick \"Allow write access\")"
    echo
    cat "$HOME/.ssh/github_deploy_hub.pub"
    echo
    read -r -p "  Press Enter once the deploy key is added... " _
  done
  ok "Hub code in $HUB_APP_DIR"
else
  ok "skipped (no Hub domain)"
fi
if [ -d "$HUB_APP_DIR/.git" ] && { [ -n "$(env_get HUB_DIR)" ] || [ "$HUB_APP_DIR" != "$(dirname
"$APP_DIR")/rendezvous_hub" ]; }; then
  env_set HUB_DIR "$HUB_APP_DIR"
  ok "docker/.env: HUB_DIR=$HUB_APP_DIR"
fi

say "8/10 OLT public keys (docker/secrets/olt_jwks)"
jwks_file="$APP_DIR/docker/secrets/olt_jwks"
jwks_check() {
  python3 - "$1" <<'PY'
import json, sys
try:
    keys = json.load(open(sys.argv[1]))["keys"]
except Exception as e:
    sys.exit(f"not a valid JWKS: {e}")
if not keys:
    sys.exit("the JWKS contains no keys")
for k in keys:
    if str(k.get("kid", "")).startswith("dev-"):
        sys.exit("still a dev key (kid starts with dev-); the production profile rejects it")
    if k.get("alg") not in ("EdDSA", "ES256") or "d" in k:
        sys.exit("keys must be public with alg EdDSA or ES256")
print(", ".join(k.get("kid", "?") for k in keys))
PY
}
if kids=$(jwks_check "$jwks_file" 2>/dev/null); then
  ok "OLT JWKS installed (kid: $kids)"
```

```
else
  echo "  Paste the JWKS (PUBLIC keys) from the OLT backend team, then press Enter and Ctrl+D:"
  tmp_jwks=$(mktemp)
  cat > "$tmp_jwks"
  if kids=$(jwks_check "$tmp_jwks"); then
    (umask 077; cat "$tmp_jwks" > "$jwks_file")
    rm -f "$tmp_jwks"
    ok "OLT JWKS saved (kid: $kids)"
  else
    rm -f "$tmp_jwks"
    die "JWKS rejected. Ask the OLT team for the public-key JWKS (docs/olt-integration.md), then run this
script again."
  fi
fi

say "9/10 DNS check"
check_dns() {
  local name=$1 resolved
  resolved=$(getent ahostsv4 "$name" | awk '!seen {print $1; seen = 1}' || true)
  [ -n "$resolved" ] || die "$name does not resolve to any IP yet. Ask the IT team to create an A record, wait,
then run again."
  echo "  $name -> $resolved   (IPs on this server: $(hostname -I | xargs))"
}
check_dns "$domain"
[ -z "$hub_domain" ] || check_dns "$hub_domain"
confirm "Is the IP above this server's public IP (or the office public IP forwarded to this server)?" \
  || die "Fix DNS first, then run this script again."

say "10/10 Starting services"
export PROFILE=single
bashrc_export PROFILE single
if [ -n "$hub_domain" ]; then
  export CS=1
  bashrc_export CS 1
fi
make up
ok "all services running"

if [ -n "$hub_domain" ] && confirm "Create the first Hub Super Admin account now?"; then
  admin_email=$(ask "Super Admin email")
  admin_name=$(ask "Super Admin full name")
  make cs-bootstrap-admin EMAIL="$admin_email" NAME="$admin_name" \
    || warn "account not created (a Super Admin may already exist). See the message above."
fi

say "Public access check"
anon_key=$(cat docker/secrets/anon_key)
health=""
for _ in $(seq 1 18); do
  health=$(curl -fsS -m 10 -H "apikey: $anon_key" "https://$domain/bridge/v1/health" 2>/dev/null || true)
  [[ $health == *'"olt_mode":"jwt"'* ]] && break
  sleep 10
done
[[ $health == *'"olt_mode":"jwt"'* ]] && ok "https://$domain/bridge/v1/health: olt_mode jwt" \
  || warn "health is not answering yet. Check the certificate: make logs s=traefik (DNS/port 80 must be open)"
code=$(curl -s -o /dev/null -m 10 -w '%{http_code}' "http://$domain/" || true)
[[ $code =~ ^30[1278]$ ]] && ok "http redirects to https ($code)" || warn "http://$domain/ answered $code
(expected a redirect)"
code=$(curl -s -o /dev/null -m 10 -w '%{http_code}' "https://$domain/" || true)
```

```
[ "$code" = 404 ] && ok "https://$domain/ 404 (only API paths are open)" || warn "https://$domain/ answered
$code (expected 404)"

cat <<EOF

Done.

  URL API (supabaseUrl) : https://$domain
  Anon key (anonKey)    : $anon_key
EOF
[ -z "$hub_domain" ] || echo "  Rendezvous Hub        : https://$hub_domain   (only from: $vpn_ranges)"
cat <<EOF

Next steps (docs/server-install.md section 5):
  1. Log in to the Hub as Super Admin and change the temporary password.
  2. Pengaturan -> Backup: choose a destination, create an encryption password, store it off the server.
  3. Pengaturan -> Alert: fill in Telegram/email, then Kirim uji.
  4. Store off the server: docker/.env, docker/secrets/, the backup password.
  5. Give the API URL and anon key to the OLT app team.
EOF
```